# Costruire figure scientifiche con Python
## Un percorso graduale con NumPy e Matplotlib

Giuseppe Mandaglio

# Indice

**1 Introduzione** **5**
1.1 Struttura dei file di dati . . . . . 5

**2 Dal grafico minimo alla figura completa** **7**
2.1 Il grafico più semplice: plot_bad.py . . . . . 7
2.2 Visualizzare punti separati: plot_bad2.py . . . . . 7
2.3 Etichetta e legenda: plot_bad3.py . . . . . 8
2.4 Nomi e unità sugli assi: plot_bad4.py . . . . . 9
2.5 Due misure nella stessa figura: plot_bad5.py . . . . . 10
2.6 Lettura compatta e salvataggio: plot_bad6.py . . . . . 10
2.7 Versione rifinita: plottos2.py . . . . . 11

**3 Fit parabolici con SciPy e NumPy** **13**
3.1 Il modello . . . . . 13
3.2 Fit con scipy.optimize.curve_fit . . . . . 13
3.3 Fit con numpy.polyfit . . . . . 14
3.4 Matrice di covarianza e stampa dei risultati . . . . . 14
3.5 Confronto tra i due approcci . . . . . 15
3.6 Risultati numerici sui dati allegati . . . . . 15

**4 Buone pratiche** **17**

# Capitolo 1

# Introduzione

Questo documento sviluppa passo dopo passo una figura scientifica in Python. Si parte da un grafico minimo e si aggiungono marker, legenda, etichette, più serie di dati, salvataggio su file, barre di errore e personalizzazione grafica. I dati vengono letti con `numpy.loadtxt`, mentre la rappresentazione grafica è affidata a `matplotlib.pyplot`.

## 1.1 Struttura dei file di dati

Nel file `datios.txt` le colonne rappresentano $x$, $y$, $\sigma_x$ e $\sigma_y$. Nel file allegato `Kter.txt` sono invece presenti $x$, $y_1$ e $y_2$. Gli script originali indicano `Kter2.dat`; nel materiale di questo documento il nome è stato uniformato a `Kter.txt`, che è il file effettivamente disponibile.

# Capitolo 2

# Dal grafico minimo alla figura completa

## 2.1 Il grafico più semplice: plot_bad.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 dati = np.loadtxt("datios.txt")
5 x = dati[:, 0]
6 y = dati[:, 1]
7 plt.plot(x, y)
8 plt.show()
```

`np.loadtxt` legge l'intera tabella. Le espressioni `dati[:,0]` e `dati[:,1]` selezionano rispettivamente la prima e la seconda colonna. Senza uno stile esplicito, `plot` collega i punti con una linea.

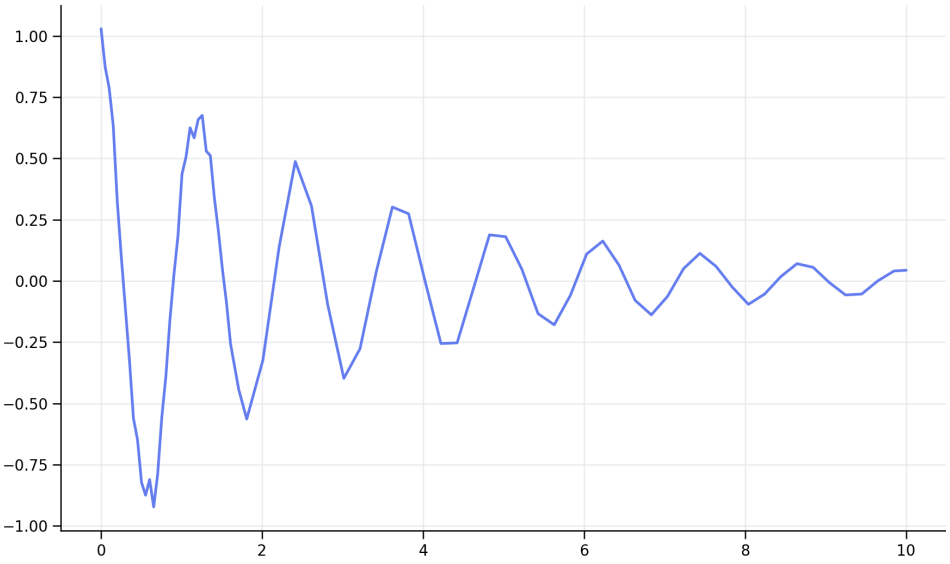



Figura 2.1: Risultato di `plot_bad.py`.

## 2.2 Visualizzare punti separati: plot_bad2.py

```
1 import numpy as np
```

```
import matplotlib.pyplot as plt

dati = np.loadtxt("datios.txt")
x = dati[:, 0]
y = dati[:, 1]
plt.plot(x, y, "o")
plt.show()
```

La stringa di formato `"o"` richiede marker circolari e, non specificando una linea, evita di collegare i punti.

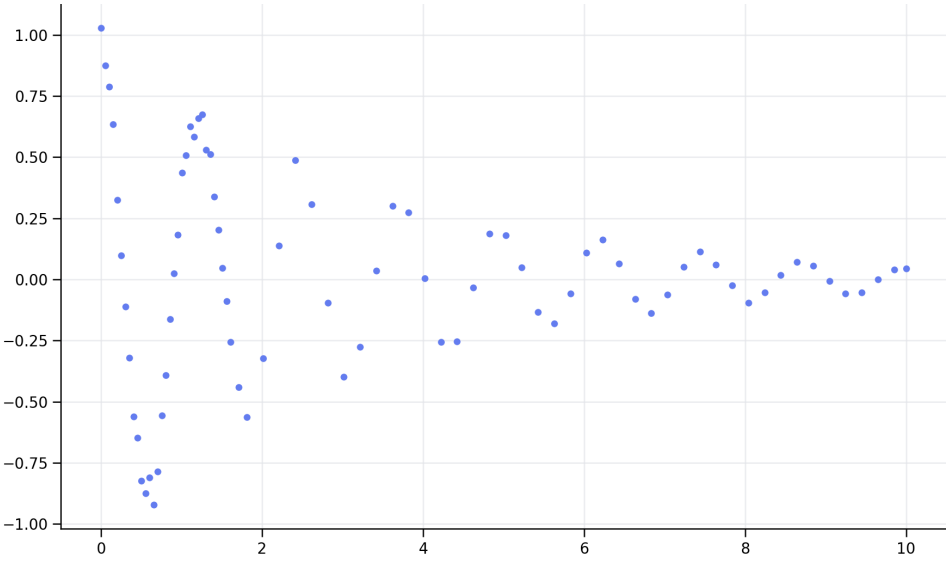



Figura 2.2: Dati rappresentati con marker circolari.

## 2.3 Etichetta e legenda: plot_bad3.py

```
import numpy as np
import matplotlib.pyplot as plt

dati = np.loadtxt("datios.txt")
x = dati[:, 0]
y = dati[:, 1]
plt.plot(x, y, "^", label="Misure")
plt.legend()
plt.show()
```

Il marker `"^"` produce triangoli. L'argomento `label` assegna un nome alla serie, mentre `legend()` crea la legenda.

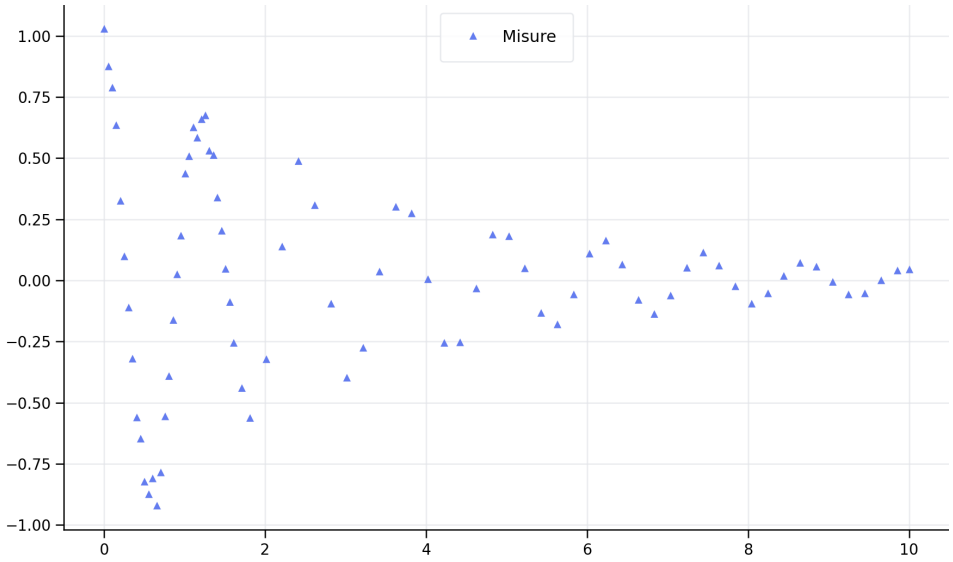



Figura 2.3: Marker triangolari e legenda.

## 2.4 Nomi e unità sugli assi: plot_bad4.py

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  dati = np.loadtxt("datios.txt")
5  x = dati[:, 0]
6  y = dati[:, 1]
7  plt.plot(x, y, "^", label="Misure")
8  plt.xlabel("Tempo (s)")
9  plt.ylabel("Distanza (cm)")
10 plt.legend()
11 plt.show()
```

Le funzioni `xlabel` e `ylabel` descrivono le grandezze riportate sugli assi. In una figura scientifica è buona pratica indicare sempre anche le unità di misura.

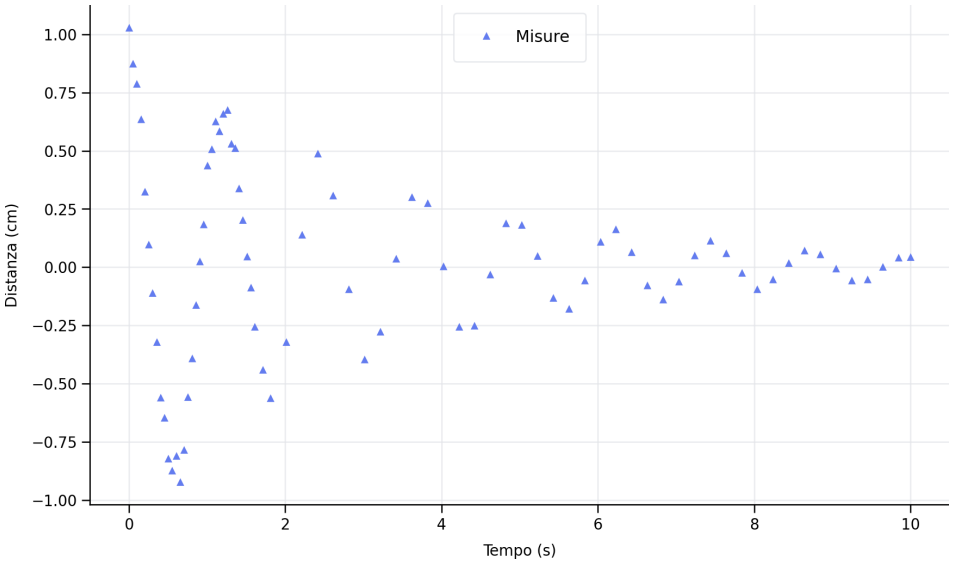



Figura 2.4: Figura con etichette degli assi.

## 2.5 Due misure nella stessa figura: plot_bad5.py

```
import numpy as np
import matplotlib.pyplot as plt

dati = np.loadtxt("Kter.txt")
x = dati[:, 0]
y1 = dati[:, 1]
y2 = dati[:, 2]
plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()
plt.show()
```

La terza colonna introduce una seconda serie. Due chiamate a `plot` disegnano le serie nello stesso sistema di assi; marker e label differenti consentono di distinguerle.

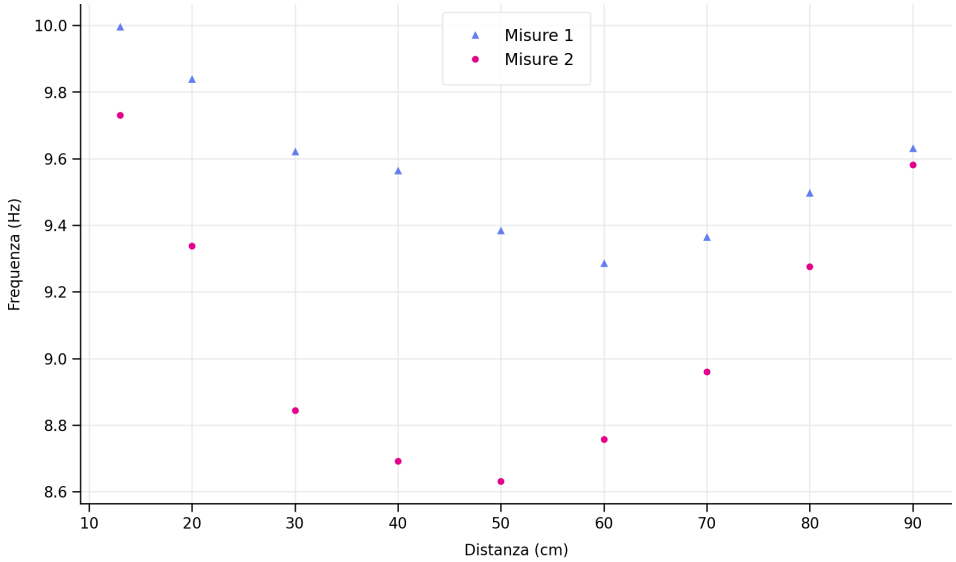



Figura 2.5: Confronto tra due serie di misure.

## 2.6 Lettura compatta e salvataggio: plot_bad6.py

```
import numpy as np
import matplotlib.pyplot as plt

x, y1, y2 = np.loadtxt("Kter.txt", unpack=True)
plt.plot(x, y1, "^", label="Misure 1")
plt.plot(x, y2, "o", label="Misure 2")
plt.xlabel("Distanza (cm)")
plt.ylabel("Frequenza (Hz)")
plt.legend()
plt.savefig("grafico.pdf")
plt.savefig("grafico.png", dpi=300)
plt.show()
```

Con `unpack=True`, ogni colonna viene restituita come array separato. Il salvataggio in PDF mantiene una grafica vettoriale; nel PNG il parametro `dpi=300` imposta una risoluzione adatta

alla stampa. È preferibile salvare prima di `show()`, perché alcuni backend possono chiudere o svuotare la figura dopo la visualizzazione.

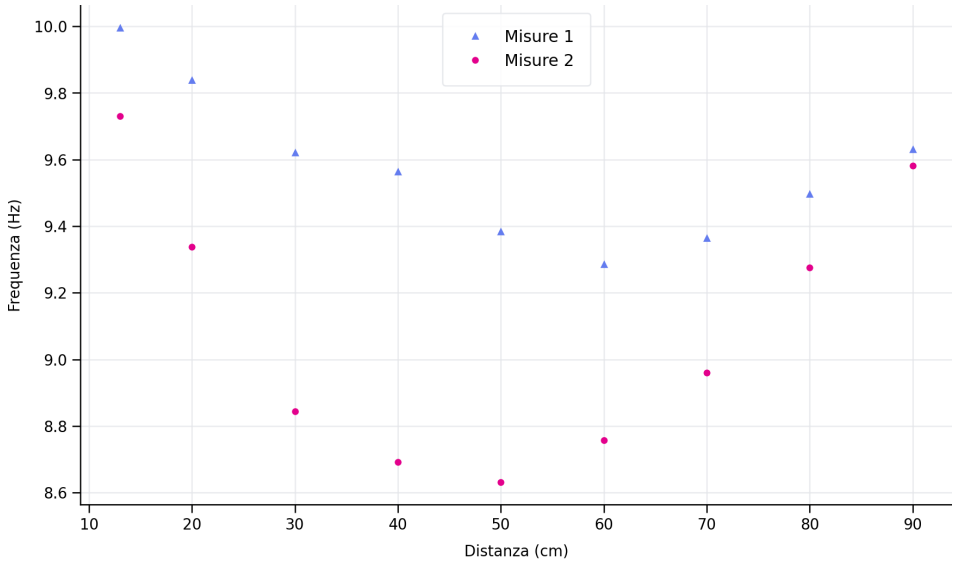



Figura 2.6: Figura a due serie salvabile in PDF e PNG.

## 2.7 Versione rifinita: plottos2.py

La versione più completa usa l'interfaccia orientata agli oggetti di Matplotlib:

```
fig, ax = plt.subplots(figsize=(8, 6))
ax.errorbar(x, y, xerr=xerr, yerr=yerr,
            fmt="o", color="blue", ecolor="gray",
            elinewidth=1, capsize=3, markersize=5,
            label="Misura")
```

`fig` rappresenta l'intera figura, mentre `ax` è il sistema di assi su cui si opera. `errorbar` accetta gli errori orizzontali e verticali. `fmt` controlla il marker, `ecolor` il colore delle barre, `elinewidth` il loro spessore e `capsize` la lunghezza dei terminali.

Le etichette sono personalizzate con `ax.set_xlabel` e `ax.set_ylabel`. `tick_params` controlla dimensione, spessore e lunghezza dei tick. `MultipleLocator` stabilisce il passo delle divisioni principali, mentre `AutoMinorLocator(5)` suddivide ogni intervallo principale in cinque parti. Gli *spines* sono i quattro bordi dell'area del grafico. Infine, `tight_layout` riduce il rischio che etichette e numeri vengano tagliati.

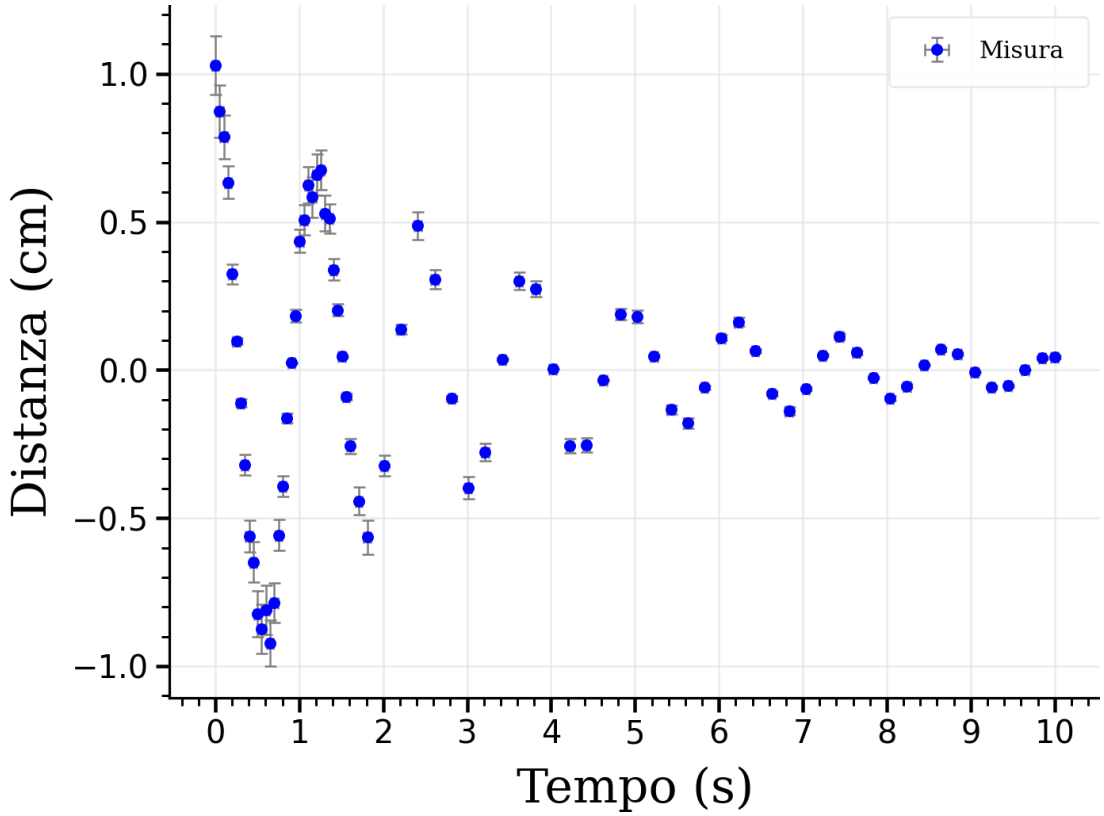



Figura 2.7: Versione rifinita con barre di errore, tick principali e secondari e stile personalizzato.

# Capitolo 3

# Fit parabolici con SciPy e NumPy

## 3.1 Il modello

Gli script assumono una parabola

$$f(x; a, b, c) = ax^2 + bx + c.$$

Il fit determina i parametri che minimizzano la somma dei quadrati dei residui. Se non si forniscono incertezze, ogni punto contribuisce con lo stesso peso.

## 3.2 Fit con scipy.optimize.curve_fit

```
from scipy.optimize import curve_fit

def parabola(x, a, b, c):
    return a*x**2 + b*x + c

popt1, pcov1 = curve_fit(parabola, x, y1)
perr1 = np.sqrt(np.diag(pcov1))
```

`curve_fit` accetta una funzione Python esplicita ed è quindi adatto anche a modelli non polinomiali. `popt1` contiene i parametri ottimali; `pcov1` è la matrice di covarianza stimata. Le incertezze standard si ricavano dalla radice quadrata degli elementi diagonali.

Per dati con errori $\sigma_y$ noti, una forma più completa è:

```
popt, pcov = curve_fit(parabola, x, y,
                       sigma=yerr,
                       absolute_sigma=True)
```

In questo caso si effettua un fit pesato e `absolute_sigma=True` interpreta gli errori come incertezze assolute.

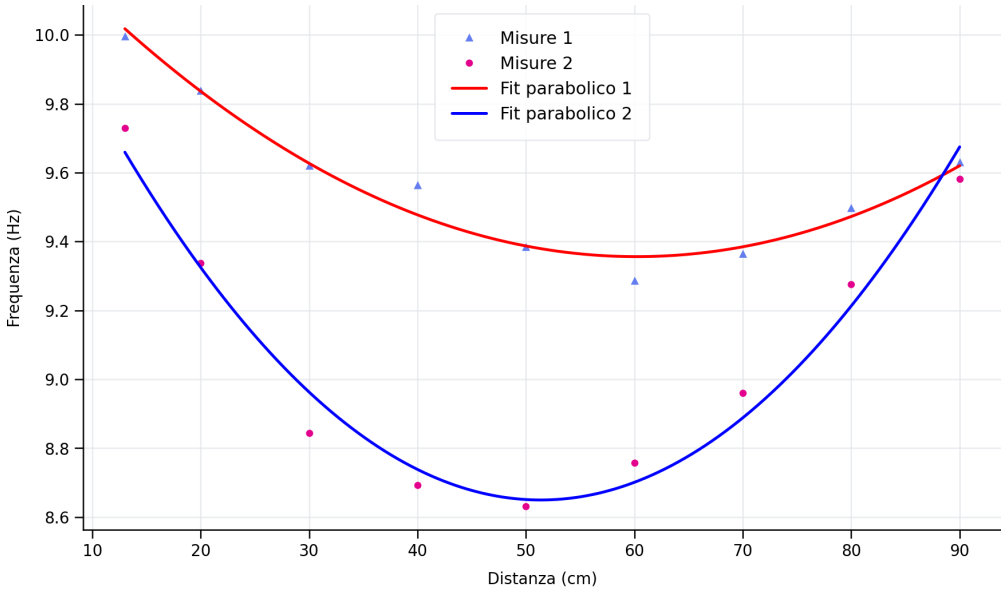



Figura 3.1: Fit delle due serie con un modello parabolico.

## 3.3 Fit con numpy.polyfit

```
p1 = np.polyfit(x, y1, 2)
yfit1 = np.polyval(p1, xfit)
```

Il grado 2 richiede un polinomio quadratico. I coefficienti sono restituiti nell'ordine $[a, b, c]$. `polyval` valuta il polinomio nei punti della griglia fitta `xfit`.

## 3.4 Matrice di covarianza e stampa dei risultati

```
p1, cov1 = np.polyfit(x, y1, 2, cov=True)
err1 = np.sqrt(np.diag(cov1))
print(f"a = {p1[0]:.5g} +/- {err1[0]:.2g}")
```

Con `cov=True`, `polyfit` restituisce anche la matrice di covarianza. La sintassi della f-string applica cinque cifre significative al valore e due all'errore. Le parentesi quadre devono rimanere all'interno dell'espressione, senza backslash.

Per un fit pesato con `polyfit`, i pesi sono proporzionali a $1/\sigma_y$:

```
p, cov = np.polyfit(x, y, 2, w=1/yerr, cov=True)
```

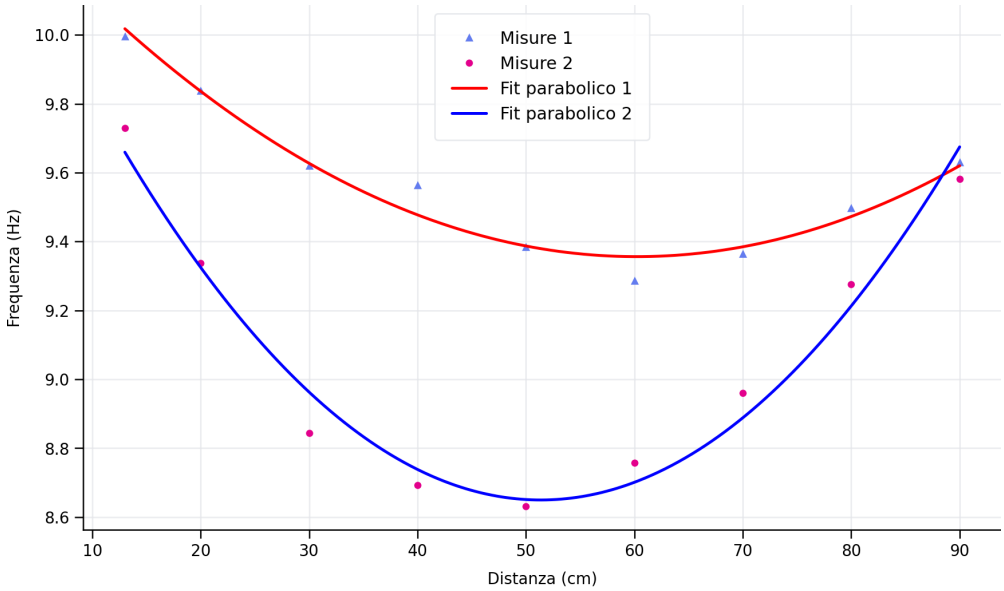



Figura 3.2: Fit parabolico costruito con `polyfit` e valutato con `polyval`.

## 3.5 Confronto tra i due approcci

| Aspetto | SciPy: curve_fit | NumPy: polyfit |
|---|---|---|
| Modello | funzione Python generica | polinomio |
| Parametri | `popt` | coefficienti |
| Covarianza | restituita direttamente | con `cov=True` |
| Pesi | `sigma=` | `w=` |
| Uso ideale | modelli generici/non lineari | regressioni polinomiali |

Per una parabola i risultati numerici dei due metodi devono essere compatibili, a parità di dati e pesi. La differenza principale è l'interfaccia: `curve_fit` descrive il modello mediante una funzione, mentre `polyfit` è specializzato nei polinomi.

## 3.6 Risultati numerici sui dati allegati

Per la prima serie si ottiene

$$a = 0.000297011 \pm 3e-05, \quad b = -0.0357571 \pm 0.0031, \quad c = 10.4323 \pm 0.068.$$

Per la seconda serie si ottiene

$$a = 0.000686333 \pm 5.2e-05, \quad b = -0.070492 \pm 0.0054, \quad c = 10.4597 \pm 0.12.$$

# Capitolo 4

# Buone pratiche

- usare nomi coerenti tra script e file di dati;
- indicare grandezza e unità su entrambi gli assi;
- distinguere le serie anche con marker diversi, non soltanto con il colore;
- salvare prima di visualizzare la figura;
- usare PDF per documenti vettoriali e PNG ad alta risoluzione per immagini raster;
- riportare parametri, incertezze e, quando appropriato, indicatori della qualità del fit;
- usare fit pesati quando le incertezze sperimentali sono note.